

HOTEL MANAGEMENT SAAS

Full Development Roadmap & Technical Specification

Version 1.0 · April 2025 · Modular Monolith → SaaS

Purpose of This Document

This document is the single source of truth for building the Hotel Management SaaS platform. It is written to be handed directly to Claude Code (or any AI coding agent) so that it can understand the full scope, make correct architectural decisions, and generate production-ready code without ambiguity.

Read every section before writing any code.

1. PROJECT OVERVIEW

Property	Value
Product name	HotelOS
Product type	Multi-tenant SaaS web application
Target users	Hotel owners, operations managers, front-desk staff, housekeeping, accounting
Architecture	Modular Monolith (designed for future microservice extraction)
Deployment target	Cloud (Azure Turkey or AWS Frankfurt — KVKK compliance)
Primary language	Turkish UI, English codebase
Mobile	Web-first (responsive); mobile app planned for later phases

1.1 Core Architectural Principles

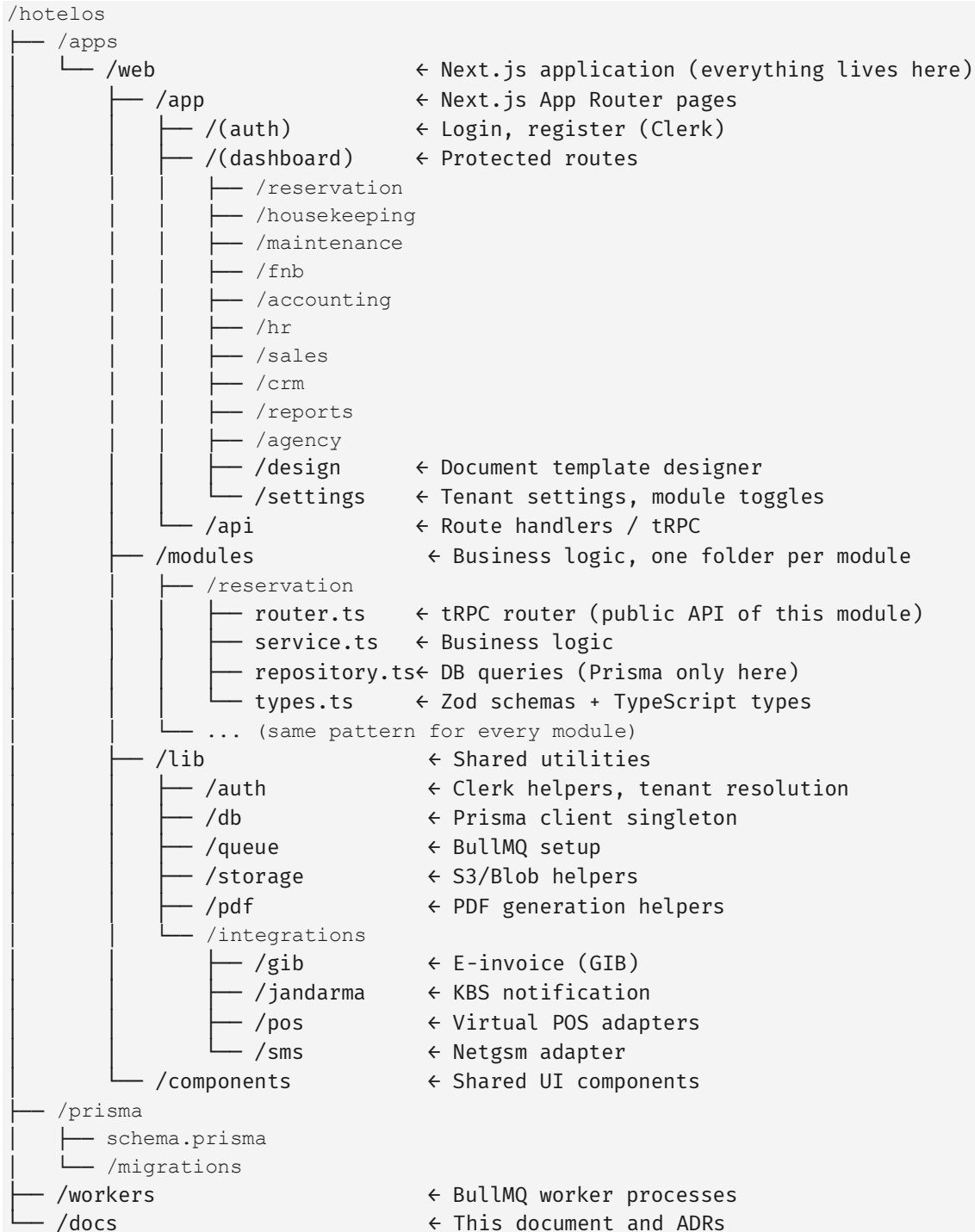
- Every module is self-contained: its own routes, services, repositories, and DB schema namespace.
- Modules communicate only through well-defined internal APIs (TypeScript interfaces) — never by importing each other's database queries directly.
- Multi-tenancy is row-level: every table that holds tenant data has a `tenant_id` column. No separate schemas per tenant.
- Feature flags control which modules a tenant (hotel) has purchased. Middleware enforces this on every request.
- All financial calculations (pricing, commissions, payroll) happen server-side only — never in the browser.

2. TECHNOLOGY STACK

Layer	Technology	Version	Reason
Frontend framework	Next.js (App Router)	14+	SSR, API routes, file-based routing, future mobile web
UI language	TypeScript	5+	Type safety across full stack
UI styling	Tailwind CSS	3+	Rapid consistent UI without custom CSS overhead
UI components	shadcn/ui	latest	Accessible, unstyled base components built on Radix UI
State management	Zustand	4+	Lightweight, no boilerplate, per-module stores
Server-side	Next.js Route Handlers + tRPC	latest	Type-safe API, no REST/GraphQL overhead for internal calls
Authentication	Clerk	latest	Multi-tenant SaaS auth, org-per-hotel, free tier available
Database	PostgreSQL	16+	Relational, JSONB for flexible fields, excellent for multi-tenant row security
ORM	Prisma	5+	Type-safe queries, migrations, schema as code
Background jobs	BullMQ + Redis	latest	Report generation, scheduled tasks, email sending
File storage	AWS S3 / Azure Blob	—	Contract PDFs, HK photos, signature files
Email / SMS	Resend (email) + Netgsm (SMS TR)	—	Transactional email; Netgsm for Turkey SMS/WhatsApp
PDF generation	Puppeteer or React-PDF	—	Invoices, contracts, payroll slips
Testing	Vitest + Playwright	—	Unit + E2E
CI/CD	GitHub Actions	—	Lint, test, deploy on push
Hosting	Vercel (frontend) + Railway or Render (API/DB)	—	Low-ops, scales automatically

3. REPOSITORY STRUCTURE

Use a single monorepo. Do not split into multiple repos until team size justifies it.



4. DATABASE SCHEMA

All tables below are in the Prisma schema. Every tenant-scoped table includes tenantId String referencing the Tenant table. Use UUIDs as primary keys everywhere.

4.1 Core / Shared Tables

Table	Key Fields	Notes
Tenant	id, name, slug, plan, activeModules[], createdAt	One row per hotel. activeModules is a string array of enabled module keys.
User	id, clerkId, tenantId, role, department, isActive	Synced from Clerk webhook. Role enum: OWNER MANAGER STAFF TECHNICIAN HK ACCOUNTING
AuditLog	id, tenantId, userId, action, entity, entityId, diff(JSON), createdAt	Every create/update/delete writes here automatically via Prisma middleware.

4.2 Reservation Module Tables

Table	Key Fields
Room	id, tenantId, number, typeId, floor, view, features(JSON), status(CLEAN DIRTY FAULTY DND)
RoomType	id, tenantId, name, capacity, basePrice
Guest	id, tenantId, firstName, lastName, tId, passportNo, phone, email, nationality, blacklisted, blacklistReason
Reservation	id, tenantId, guestId, roomId, agencyId, checkIn, checkOut, adults, children, status(WAITING CONFIRMED CHECKEDIN CHECKEDOUT CANCELLED NO SHOW), totalPrice, notes, specialRequests
ReservationPayment	id, reservationId, amount, method(CASH WIRE VIRTUALPOS PHYSICALPOS), paidAt, reference
ElectronicSignature	id, reservationId, type(KVKK CONTRACT), signedAt, fileUrl, ipAddress

4.3 Housekeeping Module Tables

Table	Key Fields
HKTask	id, tenantId, roomId, assignedUserId, type(ROOM COMMON_AREA), status(PENDING IN_PROGRESS DONE), startedAt, completedAt, durationMinutes
HKPhoto	id, taskId, url, takenAt
HKChecklist	id, tenantId, roomTypeId, items(JSON array of task names)
CommonArea	id, tenantId, name, cleaningFrequencyDays
CommonAreaSchedule	id, commonAreaId, scheduledDate, completedAt, assignedUserId
LostAndFound	id, tenantId, roomId, description, foundBy, foundAt, returnedAt, guestId

Table	Key Fields
LinenTracking	id, tenantId, roomId, sentCount, returnedCount, date

4.4 Maintenance Module Tables

Table	Key Fields
FaultReport	id, tenantId, location, description, reportedBy, assignedTo, priority(URGENT NORMAL LOW), status(OPEN IN_PROGRESS DONE CANNOT_FIX), cannotFixReason, resolvedAt
Equipment	id, tenantId, name, category, serialNo, purchaseDate, warrantyExpiry, lastServiceDate
PreventiveMaintenance	id, equipmentId, scheduledDate, completedAt, technicianId, notes
ExternalServiceFirm	id, tenantId, name, phone, specialty
ServiceVisit	id, tenantId, firmId, visitDate, description, invoiceId

4.5 Accounting Module Tables

Table	Key Fields
StockItem	id, tenantId, name, unit, category, barcode, currentStock, minStock, lastPrice, vatRate
Warehouse	id, tenantId, name, type(MAIN DEPARTMENT SUB), parentId
Invoice	id, tenantId, type(PURCHASE SALE), supplierId, warehouseId, date, totalAmount, vatAmount, gibStatus, gibEnvelopId, lines(JSON)
DepartmentRequest	id, tenantId, requestedBy, warehouseId, status(PENDING APPROVED REJECTED MODIFIED), items(JSON), note
WarehouseTransfer	id, tenantId, fromWarehouseId, toWarehouseId, items(JSON), transferredAt, transferredBy
FixedAsset	id, tenantId, name, purchasePrice, purchaseDate, depreciationYears, currentValue
CashAccount	id, tenantId, name, type(CASH BANK VIRTUALPOS PHYSICALPOS), balance
Transaction	id, tenantId, accountId, amount, direction(IN OUT), category, reference, relatedInvoiceId, date

4.6 HR Module Tables

Table	Key Fields
Employee	id, tenantId, userId, firstName, lastName, tcId, phone, department, position, startDate, endDate, isActive, contractType
Shift	id, tenantId, employeeId, date, startTime, endTime, type(NORMAL OVERTIME HOLIDAY)
LeaveRequest	id, tenantId, employeeId, type(ANNUAL EXCUSED UNPAID), startDate, endDate, status(PENDING APPROVED REJECTED), approvedBy

Table	Key Fields
Payroll	id, tenantId, employeeId, month, year, baseSalary, overtime, bonus, deductions, grossSalary, netSalary, SGKEmployee, SGKEmployer, incomeTax
DisciplineRecord	id, tenantId, employeeId, type, description, date, documentUrl
Training	id, tenantId, employeeId, name, completedAt, expiryDate, certificateUrl

4.7 Agency & Pricing Tables

Table	Key Fields
Agency	id, tenantId, name, commissionRate, extraDiscount, kickbackThreshold, kickbackRate, contractUrl
PriceCalendar	id, tenantId, agencyId(nullable), startDate, endDate, pricePerNight, createdBy, createdAt
MultiplierMatrix	id, tenantId, persons, multiplier
ReservationPriceCheck	id, reservationId, expectedPrice, actualPrice, isCorrect, checkedAt

4.8 CRM & Survey Tables

Table	Key Fields
Survey	id, tenantId, reservationId, guestId, ratings(JSON), comment, submittedAt, isCheckoutSurvey
ExternalReview	id, tenantId, guestId(nullable), platform, reviewerName, rating, comment, reviewDate, sourceUrl
GuestComplaint	id, tenantId, guestId, reservationId, description, status(OPEN RESOLVED), resolvedAt, resolvedBy
GuestSegment	id, tenantId, name, rules(JSON)

5. MODULE SPECIFICATIONS

Each module follows the same file pattern: *router.ts* (tRPC) → *service.ts* (business logic) → *repository.ts* (Prisma queries). Below are the required endpoints and business rules for each module.

5.1 Reservation Module

Key Business Rules

- When a reservation is created, automatically calculate totalPrice using: $(\text{multiplier} \times \text{nights} \times \text{pricePerNight}) \times (1 - \text{agencyDiscount})$.
- If the room status is FAULTY or DIRTY, block check-in and show a warning.
- No-show and cancellation policies are defined per tenant in Settings. If a reservation passes the no-show threshold, trigger a fee transaction automatically.
- Electronic signature: generate a unique link, send via WhatsApp/email/SMS. Store signed document URL. Mark reservation as signed. If not signed digitally, staff can upload a scanned copy.
- Jandarma KBS notification must be sent within the legally required time window after check-in. Use a BullMQ delayed job.
- Guest history tab: aggregate all past reservations for this guest, total revenue, total profit (revenue minus per-person cost), extra services cost.

Required tRPC Endpoints		
Endpoint	Input	Returns
reservation.list	tenantId, filters(status, dateRange, guestName, agencyId)	Paginated reservation list with guest and room info
reservation.get	id	Full reservation with payments, signatures, room, guest
reservation.create	ReservationCreateInput (see types.ts)	Created reservation with calculated price
reservation.update	id, ReservationUpdateInput	Updated reservation
reservation.cancel	id, reason	Updated reservation (status=CANCELLED), triggers fee if policy applies
reservation.checkIn	id	Updated reservation (status=CHECKEDIN), triggers KBS job, updates room status to DIRTY
reservation.checkOut	id	Updated reservation (status=CHECKEDOUT), requires survey completion flag
reservation.sendSignatureLink	id, channel(WHATSAPP EMAIL SMS)	Sent confirmation
reservation.pricePreview	guestCount, checkIn, checkOut, agencyId?	Calculated price breakdown

room.list	tenantId, filters(status, type, floor)	Room list with current status
room.updateStatus	id, status, faultDetail?	Updated room
guest.search	tenantId, query	Matching guests for autocomplete
guest.get	id	Full guest profile with history and CRM data

5.2 Housekeeping Module

Key Business Rules

- Room approval (marking CLEAN) requires the minimum number of photos defined for that room type. The system must reject approval if photo count is below the threshold.
- When a room is marked FAULTY from HK app, a FaultReport is automatically created in the Maintenance module with location pre-filled.
- Cleaning duration is measured from task startedAt to completedAt. Include in HK report.
- Common areas have a cleaningFrequencyDays value. A scheduled BullMQ job runs daily at 06:00 and creates pending CommonAreaSchedule entries for any area whose last completion is older than its frequency.
- Automatic HK report is generated at a tenant-configurable time (default 09:00) and sent by email to the manager. Report includes: room statuses summary, tasks completed/pending, average cleaning time, common area status.

Required tRPC Endpoints

Endpoint	Notes
hk.taskList	Filter by assignedUser, status, date, room
hk.startTask	Sets startedAt, status=IN_PROGRESS
hk.completeTask	Validates photo count. Sets completedAt, duration. Updates room status.
hk.addPhoto	Upload to S3, create HKPhoto record
hk.lostAndFound.create / list / resolve	Full CRUD for lost items
hk.schedule.list	Today's common area schedule
hk.schedule.complete	Mark common area task done
hk.report.generate	Generate and return (or email) the HK daily report
hk.linen.log	Record sent/returned linen counts per room

5.3 Maintenance Module

Key Business Rules

- Cannot-fix reason is mandatory when status is set to CANNOT_FIX.
- Fault location description is mandatory (minimum 20 characters).

- Priority URGENT faults must appear highlighted (red) on the dashboard and trigger an in-app notification to the assigned technician immediately.
- Preventive maintenance: a BullMQ job checks daily for equipment whose nextServiceDate is within 7 days and sends an alert to the maintenance manager.
- Equipment warranty expiry: alert sent 30 days before expiry.

Required tRPC Endpoints

Endpoint	Notes
maintenance.fault.list	Filter by status, priority, location, dateRange
maintenance.fault.create / update / delete	Full CRUD. On create from HK, location is pre-filled.
maintenance.equipment.list / create / update	Full CRUD. Include warranty status computed field.
maintenance.preventive.list / complete	Schedule list and completion recording
maintenance.serviceFirm.list / create	External firm registry
maintenance.serviceVisit.create / list	Log of visits per firm
maintenance.report.unresolved	List of cannot-fix faults with reasons, for reporting module

5.4 Accounting Module

Key Business Rules

- All prices stored in Turkish Lira (TRY) as integers (kuruş / cents) to avoid floating point errors. Display with /100.
- GIB integration: use an intermediary integration library (e.g. edonusum or similar). Never implement raw GIB XML from scratch. The GIB envelope ID must be stored on every e-invoice.
- Warehouse stock is updated in a database transaction together with the invoice line items. Never update stock without an invoice/transfer record.
- Department requests follow a strict status machine: PENDING → APPROVED (stock deducted) | REJECTED | MODIFIED (quantity adjusted, then back to PENDING for re-approval).
- Every Transaction record must reference either an invoiceId, a reservationPaymentId, or a manual entry reason. Orphaned transactions are not allowed.

Required tRPC Endpoints

Endpoint	Notes
accounting.stock.list / create / update / delete	Stock item management with barcode support
accounting.warehouse.list / create	Warehouse and sub-warehouse hierarchy
accounting.invoice.list / create / update / delete	Purchase and sale invoices. Triggers stock update in same transaction.
accounting.invoice.syncGIB	Pull incoming e-invoices from GIB inbox, create Invoice records
accounting.departmentRequest.create / approve / reject / modify	Full request workflow
accounting.transfer.create / list	Inter-warehouse transfers
accounting.transaction.list / create	Cash flow entries
accounting.cashFlow.forecast	Next 90-day cash flow projection based on reservations + scheduled payables
accounting.fixedAsset.list / create / update	Fixed asset register with depreciation calculation
accounting.vatSummary	Quarterly VAT base and payable summary

5.5 HR Module

Key Business Rules

- Duplicate employee detection: when creating an employee, check tcId uniqueness within the tenant. If a match exists, show the existing record and ask the user to confirm or cancel.
- Shift conflict check: when assigning a shift, verify the employee has no overlapping shift on the same date. Return a validation error if conflict found.
- Payroll calculation formula: $\text{grossSalary} = \text{baseSalary} + \text{overtime} + \text{bonus} - \text{deductions}$.
 $\text{sgkEmployee} = \text{grossSalary} \times 0.14$ (update rate from Settings). $\text{sgkEmployer} = \text{grossSalary} \times$

0.20.5. incomeTax calculated via progressive bracket (store brackets in Settings, not hardcoded).
 $\text{netSalary} = \text{grossSalary} - \text{sgkEmployee} - \text{incomeTax}$.

- Jandarma KBS for employees: send entry/exit notifications via the same integration used for guests, but with employee identity data.
- Training expiry alerts: BullMQ job checks daily, sends alert 30 days before certificate expiry.

Required tRPC Endpoints

Endpoint	Notes
hr.employee.list / create / update / deactivate	Full lifecycle. Duplicate check on create.
hr.shift.list / create / delete	Shift management with conflict validation
hr.leave.request / approve / reject / list	Leave workflow
hr.payroll.calculate	Preview payroll for employee × month. Returns full breakdown.
hr.payroll.finalize / list	Lock and store payroll. Generate payslip PDF.
hr.discipline.create / list	Discipline records with document upload
hr.training.create / list	Certification tracking with expiry
hr.report.cost	Department-level cost breakdown (salary + SGK + bonus)

5.6 Sales, Collections & Refunds Module

Key Business Rules

- Virtual POS: store encrypted POS credentials per tenant in Settings. Build a POS adapter interface — each bank (Garanti, İşbank, Yapı Kredi, Akbank, etc.) implements the same interface. This allows adding new banks without touching business logic.
- Payment link: generate a JWT-signed URL containing reservationId and amount. The link renders a payment page that calls the tenant's virtual POS. Store result as ReservationPayment.
- Refund records must always store both guestRequestDate and refundProcessedDate separately.
- Guest marketing export: query all guests with email or phone for the tenant, format as CSV compatible with Facebook Custom Audiences and Google Customer Match. Export only with KVKK consent flag = true.
- Package products: a Package links multiple StockItems or services to a single sale price. When a package is sold, individual components are deducted from stock.

Required tRPC Endpoints

Endpoint	Notes
sales.pos.configure	Save/update virtual POS credentials for tenant
sales.payment.createLink	Generate payment link for reservation
sales.payment.processManual	Record cash / wire transfer / physical POS payment
sales.payment.list	All payments for a reservation or date range
sales.refund.create / update / list	Refund management with dual-date tracking
sales.package.create / list / assign	Package product management
sales.guestExport.marketing	Generate KVKK-compliant CSV for ad platforms
sales.priceHistory.list	Historical pricing per night and period
sales.campaign.create / list / apply	Discount campaigns by date or channel

5.7 Agency Contract & Pricing Module

Key Business Rules

- Price formula: $\text{finalPrice} = \text{multiplier}(\text{persons}) \times \text{nights} \times \text{pricePerNight}(\text{dateRange}) \times (1 - \text{agencyDiscount\%})$. This formula must be the single source of truth — used in reservation creation, price preview, and price audit report.
- Kickback auto-calculation: after each reservation payment is recorded, check if the agency's cumulative confirmed revenue for the current season exceeds the kickbackThreshold. If yes, apply kickbackRate to the excess amount and create a payable transaction.
- Price audit: for each reservation in a date range, recalculate the expected price using the formula and compare to the stored totalPrice. Export result as Excel with columns: reservationId, guestName, agency, storedPrice, expectedPrice, isCorrect(YES/NO), difference.
- All price changes are logged in AuditLog with the old and new value, timestamp, and userId.

Required tRPC Endpoints	
Endpoint	Notes
agency.list / create / update	Agency registry with contract upload to S3
agency.priceCalendar.set	Bulk or individual price per date range
agency.multiplier.set	Set multiplier per person count
agency.pricePreview	Given persons + dates + agencyId, return full price breakdown
agency.priceAudit	Run price correctness check, return/export results
agency.kickback.calculate	Calculate kickback owed to agency for period
agency.commissionReport	Agency-wise commission totals and % of revenue

5.8 CRM, Surveys & Guest Relations Module

Key Business Rules

- Checkout survey gate: when staff attempts to check out a reservation, the system checks if an `isCheckoutSurvey=true` Survey exists for that `reservationId`. If not, the checkout is blocked and the survey form is shown. This gate can be disabled per tenant in Settings.
- External review import: staff provides URLs for HolidayCheck, TripAdvisor, Google, etc. in Settings. A background job scrapes or uses available APIs to pull reviews. If the platform reviewer name does not match any guest, staff can manually link to a Guest record.
- Complaint book OCR: accept a photo upload, run through an OCR service (Azure Vision or Tesseract), extract text, auto-detect language, translate to Turkish if foreign. Pre-fill complaint form with extracted text. Staff can edit before saving.
- Guest birthday reminders: BullMQ job runs daily, finds guests with birthday tomorrow or today, sends personalized message via SMS or email.
- Blacklist check: on every new reservation creation, check if the guest's `tcId` or `passportNo` is on the tenant's blacklist. If yes, show a blocking warning with the reason.

5.9 Reporting Module

All Required Reports

Report Name	Data Source	Format
Daily cash summary	Transactions (all payment methods)	Screen + PDF
Reservation collections	ReservationPayment	Screen + Excel
Occupancy rate (daily/weekly/monthly/seasonal)	Reservation	Screen + Excel
RevPAR & ADR	Reservation + Room count	Screen
Channel distribution	Reservation (agencyId / direct)	Screen + Excel
Year-over-year comparison	Reservation	Screen
Agency commissions & revenue share	Agency + Reservation	Screen + Excel
Guest acquisition cost	Agency commissions + ad spend (manual entry) + HR cost (sales dept)	Screen + Excel
Per-guest / per-room profitability	Revenue - (food cost + energy cost + staff cost per person)	Screen
Warehouse stock movement	Invoice + Transfer	Screen + Excel
Staff cost by department	Payroll + SGK	Screen + Excel
Upcoming payables & receivables	Transaction + Payroll	Screen
Refund analysis (avg resolution time, reason categories)	Refund	Screen + Excel
Unresolved faults report	FaultReport (CANNOT_FIX)	Screen + Excel
HK daily report	HKTask + HKPhoto	PDF emailed
Preventive maintenance schedule	PreventiveMaintenance	Screen + PDF

Price audit report	Reservation vs formula	Excel
CRM satisfaction overview	Survey + ExternalReview	Screen
Yield management (occupancy vs price over time)	Reservation + PriceCalendar	Screen (chart)

6. MULTI-TENANCY IMPLEMENTATION

Concern	Implementation
Tenant identification	Every API request resolves tenantId from the Clerk organization ID. Helper: lib/auth/getTenantId(clerkOrgId). Store in request context.
Data isolation	All repository functions receive tenantId as the first argument and always include WHERE tenantId = \$1. No exceptions.
Module access control	Middleware checks Tenant.activeModules before routing to any module router. Return 403 if module not active for tenant.
Tenant onboarding	When a new Clerk organization is created (webhook), create a Tenant row with a default set of activeModules. Trigger a setup wizard on first login.
Tenant settings	A Settings table stores per-tenant config (no-show policy, HK report time, SGK rates, survey gate toggle, etc.). Always loaded into request context.
Super admin	A separate SUPERADMIN role (Clerk metadata) can access all tenants for support purposes. Build a /admin panel scoped to this role.

7. EXTERNAL INTEGRATIONS

7.1 GIB E-Invoice / E-Dispatch

- Use a Turkish e-invoice integration middleware library (do not implement raw GIB XML/SOAP).
- Implement an adapter pattern: GIBAdapter interface with methods: sendInvoice, getInboxInvoices, getStatus.
- Store gibEnvelopeld on every invoice. Poll status via BullMQ job every 30 minutes for pending invoices.
- Barcode scanning: accept input from a USB barcode reader (standard keyboard input on a focused input field). On scan, look up StockItem by barcode and add to current invoice/request form.

7.2 Jandarma KBS (Identity Notification System)

- Integrate via the official e-Government API gateway or an approved intermediary service.
- Guest check-in notification: triggered as a BullMQ job 5 minutes after checkIn is recorded.
- Employee entry/exit: triggered immediately on shift start/end record.
- Store notification status and response code on each Reservation / Shift record.
- Failed notifications must be retried (max 3 times with exponential backoff) and logged.

7.3 Virtual POS (Sanal POS)

- Build a VirtualPOSAdapter interface: { charge(amount, cardToken, orderId), refund(transactionId, amount), getStatus(orderId) }
- Implement one concrete adapter per bank. Start with at minimum: Garanti BBVA (Paycore), İşbank (Sanalpos), Yapı Kredi (Posnet).
- Payment link flow: POST /api/payment/[token] renders a Next.js page with 3D Secure redirect. On callback, update ReservationPayment record and redirect to success/fail page.
- Never store raw card numbers. Only store bank transaction IDs.

7.4 SMS / WhatsApp (Netgsm)

- Use Netgsm HTTP API for SMS. For WhatsApp, use Netgsm's WhatsApp Business API integration.
- All outbound messages go through a MessageService that queues via BullMQ (never send synchronously in a request handler).
- Message templates: stored in DB per tenant (customizable). Variables: {guestName}, {checkInDate}, {signatureLink}, {paymentLink}, etc.

7.5 OCR for Complaint Book

- Use Azure Cognitive Services (Computer Vision Read API) or Google Vision API for photo-to-text.
- Language detection + translation: use Azure Translator or DeepL API.
- OCR result is always editable by staff before saving — it is a suggestion, not a final value.

8. AUTHENTICATION & AUTHORIZATION

8.1 Clerk Setup

- One Clerk Application for the entire SaaS platform.
- Each hotel = one Clerk Organization. Users are members of their hotel's organization.
- Clerk Organization metadata stores: tenantId (our DB), plan, activeModules (mirror of DB for quick JWT access).
- Use Clerk's built-in roles within organizations: admin, manager, staff. Map to our Role enum in User table.

8.2 Permission Matrix

Action	OWNER	MANAGER	ACCOUNTING	HR_STAFF	HK_STAFF	TECHNICIAN
View all modules	✓	✓	–	–	–	–
Manage tenant settings	✓	–	–	–	–	–
Create / cancel reservation	✓	✓	–	–	–	–
Process payments	✓	✓	✓	–	–	–
Manage invoices & stock	✓	✓	✓	–	–	–
Manage payroll & HR	✓	✓	–	✓	–	–
HK task management	✓	✓	–	–	✓	–
Fault reports	✓	✓	–	–	✓	✓
View all reports	✓	✓	–	–	–	–
Super admin panel	SUPERADMIN only	–	–	–	–	–

9. BACKGROUND JOBS (BuIMQ)

Job Name	Trigger	Action
kbs.notifyGuest	After checkIn recorded (+5 min delay)	Send Jandarma KBS notification for guest
kbs.notifyEmployee	On shift start/end	Send Jandarma KBS notification for employee
hk.dailyReport	Cron: tenant-configurable time (default 09:00)	Generate and email HK daily report
hk.commonAreaSchedule	Cron: 06:00 daily	Create pending tasks for overdue common areas
maintenance.warrantyAlert	Cron: 08:00 daily	Alert for equipment with warranty expiring in 30 days
maintenance.serviceAlert	Cron: 08:00 daily	Alert for equipment due for service in 7 days
hr.trainingExpiry	Cron: 08:00 daily	Alert for certifications expiring in 30 days
hr.birthdayAlert	Cron: 07:00 daily	Send birthday messages to guests
sales.paymentLink.expire	After link creation (+48h delay)	Mark payment link as expired
accounting.gibStatus	Cron: every 30 min	Check GIB status for pending e-invoices
message.send	On enqueue (immediate)	Send SMS/WhatsApp/email via provider API
report.generate	On demand (user triggered)	Heavy report generation, result stored in S3 + notified

10. PHASED DEVELOPMENT ROADMAP

□ **Note:** Build phases in strict order. Do not start Phase 2 until Phase 1 is fully tested. Each phase ends with a working, deployable product.

Phase 1: Foundation (Weeks 1–4)

Goal: A working multi-tenant app skeleton where a hotel can log in, configure rooms, and create a reservation with price calculation.

- Repo setup: Next.js 14 App Router + TypeScript + Tailwind + shadcn/ui + Prisma + PostgreSQL
- Clerk integration: org-based multi-tenancy, user sync webhook, role mapping
- Tenant onboarding flow: create org → create Tenant row → setup wizard (hotel name, room types, rooms)
- Core DB schema: Tenant, User, Room, RoomType, Guest, Reservation, AuditLog
- Reservation module: full CRUD, price calculation, reservation list with filters
- Room status management: CLEAN / DIRTY / FAULTY / DND with fault detail
- Guest module: create, search, profile with reservation history
- Audit log middleware: auto-log all mutations via Prisma middleware
- Basic dashboard: occupancy summary, today's arrivals/departures

Phase 2: Payments & Agency (Weeks 5–7)

Goal: Hotels can collect payments, generate payment links, and manage agency pricing.

- Virtual POS adapter interface + first bank implementation (Garanti BBVA)
- Payment link generation and 3D Secure callback flow
- Manual payment recording (cash, wire, physical POS)
- Refund management with dual-date tracking
- Agency module: CRUD, commission/discount/kickback config
- Price calendar: date-range pricing, multiplier matrix
- Price preview endpoint (used in reservation creation)
- Electronic signature: link generation, signing page, storage
- WhatsApp/SMS/email sending via Netgsm (payment links, signature links)

Phase 3: Operations (Weeks 8–12)

Goal: Full operational modules: HK, Maintenance, F&B basics.

- HK module: task creation, assignment, photo upload, completion with photo count validation
- HK common area scheduling (BullMQ daily cron)
- HK daily report generation and email delivery
- Lost & found module
- Linen tracking
- Maintenance module: fault reports (full CRUD, priority, cannot-fix workflow)
- Equipment registry with warranty tracking
- Preventive maintenance scheduling
- External service firm registry and visit log
- F&B: menu management, allergen tagging, auto-name-tag generation
- KBS integration: guests (check-in) and employees

Phase 4: Accounting & HR (Weeks 13–18)

Goal: Full accounting and HR modules. Most complex phase — allocate extra time.

- ☐ Stock item management with barcode support
- ☐ Warehouse hierarchy (main + department + sub)
- ☐ Purchase and sale invoice management
- ☐ GIB e-invoice integration (inbox sync + outbox send)
- ☐ Department request workflow (PENDING → APPROVED/REJECTED/MODIFIED)
- ☐ Inter-warehouse transfer slips
- ☐ Fixed asset register with depreciation
- ☐ Cash account management and transaction ledger
- ☐ VAT summary report
- ☐ HR: employee lifecycle, duplicate detection
- ☐ Shift management with conflict detection
- ☐ Leave request workflow
- ☐ Payroll calculation engine (configurable SGK/tax rates)
- ☐ Payslip PDF generation
- ☐ Discipline records and training/certificate tracking

Phase 5: CRM, Reporting & Design (Weeks 19–23)

Goal: Guest intelligence, all reports, document designer.

- ☐ Checkout survey gate (blocking + bypassable)
- ☐ External review import (link-based, manual linking to guests)
- ☐ Complaint book: photo upload → OCR → translate → pre-fill
- ☐ Guest segmentation (VIP, first-timer, loyal)
- ☐ Guest blacklist with reservation blocking
- ☐ Birthday/special day reminders (BullMQ cron)
- ☐ All 19 reports from Section 5.9 (screen views)
- ☐ Excel / PDF export for all reports
- ☐ Yield management chart (occupancy vs price trend)
- ☐ Document designer: invoice, payslip, contract, HK report, name tag templates
- ☐ Price audit report with Excel export
- ☐ Marketing export (Facebook/Google compatible CSV)

Phase 6: Polish, Security & Launch (Weeks 24–26)

Goal: Production-ready: hardened, tested, documented.

- ☐ End-to-end tests (Playwright) for all critical flows
- ☐ Unit tests (Vitest) for all service and calculation functions
- ☐ Performance: add database indexes on all foreign keys and filter columns
- ☐ Rate limiting on all API routes (Upstash Redis)
- ☐ KVKK compliance audit: consent flags, data export, right-to-erasure endpoint
- ☐ Super admin panel: tenant list, module toggle, impersonation
- ☐ Billing integration (Stripe or lyzico) for module subscription management
- ☐ Onboarding documentation for hotel staff
- ☐ CI/CD pipeline: GitHub Actions → Vercel + Railway
- ☐ Monitoring: Sentry for errors, Axiom or Grafana for logs

11. CODING CONVENTIONS FOR CLAUDE CODE

Convention	Rule
File naming	kebab-case for files and folders. e.g. reservation-service.ts
Function naming	camelCase. Service functions: verb + noun. e.g. createReservation, calculatePrice
Error handling	Never use try/catch in route handlers. Use a global tRPC error formatter. Throw TRPCError with appropriate code (BAD_REQUEST, NOT_FOUND, FORBIDDEN).
Validation	All input validated with Zod schemas defined in types.ts of each module. Share schemas between client and server.
Money	Always store as integer (kuruş). Never use float for money. Use a formatCurrency(kuruş: number): string utility for display.
Dates	Store all dates as UTC in PostgreSQL. Use Day.js for manipulation. Display in Europe/Istanbul timezone.
Tenant context	Every tRPC procedure must call ctx.requireTenant() at the top. This throws UNAUTHORIZED if tenantId is missing.
Database queries	All in repository.ts files. No raw SQL in service or router files. Use Prisma's typed query builder.
Logging	Use a structured logger (Pino). Log every mutation with tenantId, userId, action, entityId. Never log PII (TC ID, passport, full name) in production logs.
Environment variables	All secrets in .env.local (never committed). Documented in .env.example with placeholder values.
Comments	Write comments for why, not what. Business rule comments (e.g. // SGK rate per 2024 regulation) are mandatory on all financial calculations.
Translations	All user-facing strings in /messages/tr.json. Use next-intl. No hardcoded Turkish text in component files.

12. FIRST COMMAND TO CLAUDE CODE

When handing this document to Claude Code, start with the following prompt:

Read this entire roadmap document before writing any code.

Then implement Phase 1 in full:

1. Initialize the Next.js 14 monorepo with TypeScript, Tailwind, shadcn/ui, Prisma, PostgreSQL.
2. Set up Clerk with organization-based multi-tenancy.
3. Create the Prisma schema for Phase 1 tables (Section 4).
4. Implement the Reservation module (router, service, repository, types).
5. Build the reservation list and create/edit pages following the folder structure in Section 3.
6. Follow all conventions in Section 11 throughout.

13. CLAUDE.md & PROGRESS TRACKING

This section defines how Claude Code must manage its own memory and track progress across sessions. Follow these rules without exception.

13.1 CLAUDE.md — Setup Instructions

When you first initialize the project repository, create a file called CLAUDE.md in the root of the repository. This file is your persistent memory across sessions. It must always exist and always be up to date.

CLAUDE.md must contain the following sections:

- **PROJECT OVERVIEW:** one-paragraph summary of what HotelOS is, the stack (Next.js, TypeScript, PostgreSQL, Prisma, Clerk, BullMQ), and the architecture (modular monolith, multi-tenant, SaaS).
- **ARCHITECTURAL RULES:** the non-negotiable rules from Section 11 of the roadmap — money as integers, UTC dates, tenantId on every query, no cross-module DB access, Zod validation on all inputs, structured logging, no PII in logs.
- **MODULE LIST:** all 10 modules with one-line description each (Reservation, HK, Maintenance, F&B, Accounting, HR, Sales, Agency, CRM, Reporting).
- **INTEGRATION SUMMARY:** GIB (e-invoice), Jandarma KBS, Virtual POS (adapter pattern), Netgsm (SMS/WhatsApp), OCR (Azure Vision).
- **CURRENT PHASE:** which phase is currently active (e.g. Phase 1 — Foundation).
- **PROGRESS TRACKER:** the full task checklist from Section 10, with each item marked as ☐ (not started), ☒ (in progress), or ☒ (done).

Keep CLAUDE.md concise. It is a quick-reference, not a full specification. The full spec is this roadmap document.

13.2 The Two Mandatory Rules Per Prompt

Rule	When	What to do
RULE 1 — Check before starting	At the very beginning of every prompt, before writing a single line of code	Open CLAUDE.md. Read the PROGRESS TRACKER section. Identify the current phase and the next uncompleted task. If the requested task conflicts with the current phase order, flag it to the user and ask for confirmation before proceeding.
RULE 2 — Mark after completing	At the very end of every prompt, after all code is written, tested, and committed	Open CLAUDE.md. Find every task that was completed in this session. Change its marker from ☐ or ☒ to ☒ . If a task is partially done, mark it ☒ . Update CURRENT PHASE if the phase changed. Save the file and include it in the same commit.

Note: These two rules are not optional. Skipping RULE 1 risks building out-of-order. Skipping RULE 2 causes the next session to lose track of what was done. Treat CLAUDE.md like a surgeon treats a pre-op checklist.

13.3 CLAUDE.md Template

Create this exact file at project root on first run. Replace placeholder values as work progresses.

```
# CLAUDE.md — HotelOS Project Memory
```

Project Overview

HotelOS is a multi-tenant SaaS hotel management platform built with Next.js 14 (App Router), TypeScript, Tailwind CSS, shadcn/ui, tRPC, Prisma, and PostgreSQL. Authentication via Clerk (org-per-tenant). Background jobs via BullMQ + Redis. Architecture: modular monolith designed for future microservice extraction. Deployment: Vercel (frontend) + Railway (DB/workers).

Architectural Rules (Non-Negotiable)

- All money stored as integers in kurus (TRY cents). Use formatCurrency() for display.
- All dates stored as UTC. Display in Europe/Istanbul timezone using Day.js.
- Every repository function receives tenantId as first arg. Always filter by it.
- Modules never import each other's repository files. Use module router interfaces only.
- All inputs validated with Zod. Schemas defined in module/types.ts, shared client+server.
- Every mutation logged to AuditLog via Prisma middleware (tenantId, userId, entity, diff).
- Never log PII (TC ID, passport, full name, card data) in production logs.
- Every tRPC procedure calls ctx.requireTenant() as first line.
- No raw SQL. Prisma query builder only in repository.ts files.

Modules

1. Reservation – rooms, guests, bookings, check-in/out, e-signature
2. Housekeeping – room/area cleaning tasks, photos, checklists, lost & found
3. Maintenance – fault reports, equipment, preventive maintenance
4. F&B – menus, allergens, recipes, food cost, name tags
5. Accounting – stock, invoices (GIB), warehouse, payables, cash flow
6. HR – employees, shifts, leave, payroll, training
7. Sales – virtual POS, payment links, refunds, packages, campaigns
8. Agency – contracts, pricing calendar, multipliers, kickback, audit
9. CRM – surveys, reviews, complaints, blacklist, segmentation
10. Reporting – all 19 reports (screen + Excel/PDF export)

Integrations

- GIB e-invoice/e-dispatch: adapter in lib/integrations/gib/
- Jandarma KBS: adapter in lib/integrations/jandarma/
- Virtual POS: VirtualPOSAdapter interface, one file per bank in lib/integrations/pos/
- Netgsm SMS/WhatsApp: adapter in lib/integrations/sms/
- OCR (complaint book): Azure Vision or Google Vision in lib/integrations/ocr/

Current Phase

Phase 1 – Foundation (in progress)

Progress Tracker

Phase 1 – Foundation

- ☐ Repo setup: Next.js 14 + TypeScript + Tailwind + shadcn/ui + Prisma + PostgreSQL
- ☐ Clerk integration: org-based multi-tenancy, user sync webhook, role mapping
- ☐ Tenant onboarding flow: create org → Tenant row → setup wizard
- ☐ Core DB schema: Tenant, User, Room, RoomType, Guest, Reservation, AuditLog
- ☐ Reservation module: full CRUD, price calculation, list with filters
- ☐ Room status management: CLEAN / DIRTY / FAULTY / DND with fault detail
- ☐ Guest module: create, search, profile with reservation history
- ☐ Audit log middleware: auto-log all mutations via Prisma middleware
- ☐ Basic dashboard: occupancy summary, today's arrivals/departures

Phase 2 – Payments & Agency

- ☐ Virtual POS adapter interface + Garanti BBVA implementation
- ☐ Payment link generation and 3D Secure callback flow
- ☐ Manual payment recording (cash, wire, physical POS)

- ☐ Refund management with dual-date tracking
- ☐ Agency module: CRUD, commission/discount/kickback config
- ☐ Price calendar: date-range pricing, multiplier matrix
- ☐ Price preview endpoint
- ☐ Electronic signature: link generation, signing page, storage
- ☐ WhatsApp/SMS/email sending via Netgsm

Phase 3 – Operations

- ☐ HK module: task creation, assignment, photo upload, completion validation
- ☐ HK common area scheduling (BullMQ daily cron)
- ☐ HK daily report generation and email delivery
- ☐ Lost & found module
- ☐ Linen tracking
- ☐ Maintenance module: fault reports (full CRUD, priority, cannot-fix workflow)
- ☐ Equipment registry with warranty tracking
- ☐ Preventive maintenance scheduling
- ☐ External service firm registry and visit log
- ☐ F&B: menu management, allergen tagging, auto-name-tag generation
- ☐ KBS integration: guests (check-in) and employees

Phase 4 – Accounting & HR

- ☐ Stock item management with barcode support
- ☐ Warehouse hierarchy (main + department + sub)
- ☐ Purchase and sale invoice management
- ☐ GIB e-invoice integration (inbox sync + outbox send)
- ☐ Department request workflow
- ☐ Inter-warehouse transfer slips
- ☐ Fixed asset register with depreciation
- ☐ Cash account management and transaction ledger
- ☐ VAT summary report
- ☐ HR: employee lifecycle, duplicate detection
- ☐ Shift management with conflict detection
- ☐ Leave request workflow
- ☐ Payroll calculation engine
- ☐ Payslip PDF generation
- ☐ Discipline records and training/certificate tracking

Phase 5 – CRM, Reporting & Design

- ☐ Checkout survey gate
- ☐ External review import
- ☐ Complaint book: photo → OCR → translate → pre-fill
- ☐ Guest segmentation
- ☐ Guest blacklist with reservation blocking
- ☐ Birthday/special day reminders
- ☐ All 19 reports (screen views)
- ☐ Excel / PDF export for all reports
- ☐ Yield management chart
- ☐ Document designer (invoice, payslip, contract, HK report, name tag)
- ☐ Price audit report with Excel export
- ☐ Marketing export (Facebook/Google CSV)

Phase 6 – Polish, Security & Launch

- ☐ End-to-end tests (Playwright) for all critical flows
- ☐ Unit tests (Vitest) for all service and calculation functions
- ☐ Database indexes on all FK and filter columns
- ☐ Rate limiting on all API routes (Upstash Redis)
- ☐ KVKK compliance audit
- ☐ Super admin panel
- ☐ Billing integration (Stripe or Iyzico)

- ❑ Onboarding documentation
- ❑ CI/CD pipeline: GitHub Actions → Vercel + Railway
- ❑ Monitoring: Sentry + Axiom/Grafana

13.4 Workflow Summary for Claude Code

Step	Action
Every prompt — Step 1	Read CLAUDE.md → check PROGRESS TRACKER → confirm what is done and what is next before touching any code.
Every prompt — Step 2	Do the work: write code, run tests, fix errors.
Every prompt — Step 3	Update CLAUDE.md: mark completed tasks ✓, partial tasks ⚙, update CURRENT PHASE if needed.
Every prompt — Step 4	Commit all code changes AND the updated CLAUDE.md in the same git commit. Commit message format: 'phase-N: short description of what was done'.
Starting a new phase	Update the CURRENT PHASE line in CLAUDE.md. Add a dated note: '## Phase N started: YYYY-MM-DD'.
If asked to do something out of order	Flag it: 'This task belongs to Phase N but Phase M is current. Shall I proceed anyway or complete Phase M first?' Wait for confirmation.
If CLAUDE.md is missing	Stop. Do not write any code. Recreate CLAUDE.md from the template in Section 13.3 of the roadmap, then proceed.